

Vision for Document Module

Sep 24, 2026 · @Bhaskar Choudhury

What this section is, and why it comes first

The Documents section is the heart of CompliBoard. Every company has requirements, policies and compliance documents; some keep them organised, most don't. Whoever holds the EHS job spends real time keeping every document compliant and every deadline met. CompliBoard takes that headache away: it reads the documents, tells the person what each one is, what's wrong with it, what dates it sets, and what to do next.

It comes first because the scanned document is the source of truth for everything after it. Audits check evidence that lives in documents. The calendar takes its dates from permits and certificates. HR reads employee handbooks and records. So the shape of what a scan produces matters more than the screen, because three sections will read it and inherit whatever is decided here.

The hook is the report. A user's files may live anywhere; CompliBoard's reports live in CompliBoard. Once a company has enough reports inside, it keeps using it.

Finish line for rev 1. A user uploads a file or a folder, sees a correct reading of each document, browses and groups their documents, opens any report in a drawer and prints it, corrects what the scan got wrong, confirms the facts the scan found, and sends the dates to the calendar. Every scan writes a cost row. An unreadable file is said out loud. The data a document exposes is written down as a contract in SCHEMA.md. All of it on production, judged against incognito Claude and free ChatGPT on the same documents.

How documents arrive

Rev 1 is upload only. A person uploads one file, several, or a whole folder from the browser. Files are stored in CompliBoard (bucket `company-documents`) and kept until the person deletes them. Uploads also arrive from a conversation in the Compliance Workspace, which already works: autosave, classification on the file card, the file travels with the conversation.

Drive connection is its own run, later. A person will connect Google Drive or OneDrive and CompliBoard will read the files in place. It never downloads them to keep, never rearranges their folders however messy they are, and stores only its own reports. Their drive folders show in CompliBoard as read-only folders. This needs Google and Microsoft developer accounts, OAuth, tokens and change detection; the owner is setting up the accounts on the side.

Every document has a source. By hand, from a conversation, or from a drive. That is a column on the document, shown in the report.

Every document belongs to a site. A permit names its facility; a handbook is company-wide. The scan proposes the site and the person confirms it. Site is a filter on the page.

Old and new are both kept. When a newer version of a document arrives, the old one stays and the new one is judged. The scan proposes the match (same kind, same issuer, same site) and asks when it isn't sure. On screen one document shows with its versions under it: "Air permit 26-2841, 3 versions". If old versions become a nuisance later, the app asks "these are older files, ok to delete?" and never deletes on its own.

Freshness has two clocks, and the nudge uses the document's own. When a file was uploaded is one date; when the document says it was revised or issued is another. A handbook uploaded yesterday with "Revised January 2023" on the cover is the one to ask about. The report says "this document is X years old; if there is a newer version, add it", with two answers: "Add a newer version" and "This is the latest". The second answer is recorded as a fact.

What a scan does: the open baseline

Rev 1 is raw Claude. The first two attempts at this product made the requirement table the spine of every answer, and every answer came out worse than incognito Claude and free ChatGPT. So rev 1 of every section runs on the model's own power plus the interface. The requirement table is not in the picture. It comes back only after the baseline holds its own, one piece at a time, each measured.

One open call per document. The file goes to the strongest model as a native document block (a PDF whole, so it reads layout, tables and scanned pages), with who the company is: name, industry, address and site, and the labels already in use for this company. The prompt says: read this, tell us what it is, what it says, what it proves, when it matters, what's wrong with it against the rules that govern it, and how sure you are. Web search is available and the model decides when to use it. No requirement rows, no switch vocabulary, no candidate list shaping what it may say.

Jurisdiction comes from the company and the document confirms it. A document often doesn't say where it applies; an SDS has none, a handbook mixes federal and state. The model is told where the company is. A permit from Oregon DEQ says Oregon; a handbook with no state named is read against the company's state. The document adds, never overrides.

Dates come out of the same call. Today an upload makes two model calls, a review and a separate date extraction; neither writes a cost row. In rev 1 one call returns everything and the calendar reads its dates from that answer. One file read once.

What the model is asked to judge depends on what the document is. A written program or policy is checked against the standard that governs it. A permit or certificate is the agency's document: nothing to fix, what matters is its dates and its conditions. A record, roster or log is evidence: what matters is completeness and recency. A supplier's document (an SDS) is theirs, not the company's.

Every saving is a switch, measured after the baseline, never built into it. Cheap model to read, strong model to judge. No search during the read. Search restricted to the agency's own site. One pass instead of several. Batch API at half price for overnight runs. Prompt caching for documents re-sent per turn. Each one measured on the golden documents, run three times, against the open baseline.

Build on Haiku, judge on Opus. `.env.local` runs Haiku; the machinery is built and tested there. When it works, the same golden documents run on Opus 5, Sonnet 5 and Haiku and the owner judges output. The cost ledger makes that a number.

The record from production. The current scan reviewed the same Emergency Action Plan five times and returned five different answers: expiry **None** three times and **2022-02-01** twice, a regulation reference on three and none on two, gaps from 7 to 10. That is why golden cases run three times. Six "misclassified" files in the export are all exactly 3,852 bytes uploaded the same day; the evidence says they are one demo FMLA handbook under six names and the model read them right. The owner confirms.

What a scan produces: the contract

This is the data shape Audits, Calendar and HR will read. It was read off the report drawer, not the other way round. It goes into SCHEMA.md before Run 1 is briefed. We do not scan to fit the schema; we change the schema when the scan output needs it.

What	Fields	Notes
What it is	kind (permit, certificate, program, policy, record, supplier document), title the scan gives it, issuer, agency or agencies, subject, site, jurisdiction,	Agency and subject are multi-valued. Kind decides which status vocabulary applies.
Status, by kind	permits and certificates: current, expiring, expired · programs and policies: no gaps found, gaps found · records: recorded, with recency · any: could not	One green word for every kind is the fabricated all-clear. "Compliant" is never a status.
The date that	expiry for a permit, revised-on for a program, last entry for a record	Shown on the row, right side.
Summary	two or three sentences, in the report's serif	Says what kind of thing it is and what that means for the reader.
Gaps	rows with ids: description, fix, citation with link, locator in the document, whether a draft is	Gaps are rows, not a text array, so a checklist made a week later can
Deadlines	title, date, source line in the document, whether it recurs, whether it is in the calendar	Sent to the calendar on the person's say-so; the calendar section owns them from there.
Conditions	what the document obliges the company to keep doing, with the condition number and the evidence	Permits and certificates mainly. Audits will read these.

Facts found	the fact, the verbatim quote, the section or page, as-of date, and one line on what it affects	Proposed to the confirmation queue, never written silently. A quote not
Fresh	the document's own age and the nudge, if any	Answered by "add a newer version"
Version	which existing document this is a version of, proposed, with the person's confirmation	Confirmed matches are facts; "not the same permit?" is always
Provenance	when it was read, which model, how many searches it ran, prompt hash, cost row id	Shown to the person as a quiet line; kept in full for us.
Readability	read, could not read with the reason, not yet read	A status the screen shows, never a log line.

Where the pieces live today, and what changes. `switch_determinations` already has the shape of a document-backed fact: `document_id`, `locator`, verbatim quote, `evidence_class`, `confidence`, `model`, `prompt_sha256`. `obligation_evidence` has the Audits shape: `valid_from`, `valid_until`, `superseded_by`. Both are empty and nothing writes them; they stay parked until the requirement table returns. For rev 1: a `documents.status` column (uploaded, reading, read, could not read); `documents.source` (by hand, conversation, drive); a `document_index` record the scan writes with the groupable columns (kind, agencies, subjects, site, status, significant date, title); gaps as their own table; `fact_proposals` gains a nullable `document_id`, a `locator` and a `source`, and `topic_id` becomes nullable; `calendar_events` gains `document_id`; a per-company label list the scan reads before it writes; a pinned-view record.

What does not need the requirement table. None of the above. The join key to requirements (`evidence_types` on the library) is empty on all 205 rows today; filling it is content work for the verification phase, and when it exists the same columns join to it.

The screen: one table, folder as a filter, group by anything

A document has properties, not a place. A folder is where a person put a file; it is set by the user and never touched by the scan. Everything else is set by the scan: agency, subject, kind, site, status, the date that matters. So a file lives in one folder and has many properties, and any property can be the one you group by. This is Excel: one table, filter narrows, group arranges. Everyone knows it. The old Windows-style folder tree goes.

Folders stay, as a promise. We do not change how people work today. They can make folders, rename them, move files into them, and drive folders will appear beside them read-only. But a folder is a filter, not a group, and the two never mix in one list. Agency rows and folder rows in the same list show the same file twice and read as a mistake.

The controls row. Folder: All files, then their folders with counts, New folder, Connect your drive. Group by: Status, Agency, Subject, Kind, Site, Folder, None. Site: All, or one site, shown only when the company has more than one. Pin this view. Add files.

Status first is the default. The operator's question walking in is "what needs me today". So the page opens grouped by status: Needs work, Expiring within 90 days, Expired, Could not read, Not yet read, then Current below. The meta line on every row prints agency and subject, so people see DEQ and OSHA without grouping by them. Agency is one click; pinned, it is their Monday view. Group by None is the flat sortable table.

Groupings are fixed; their values emerge. The column names never change. The values come from the documents. A cannabis grower's first OLCC licence creates an OLCC group; a food manufacturer gets FDA, ODA and the county health department; a chemical distributor gets DEQ, OSHA, DOT and the fire marshal. Nobody configures it. Subject does the same: hazardous waste, worker safety, licensing, water. Two axes, agency and subject, cover how an EHS person thinks and how an owner thinks.

The trap in emergent labels, and the fix. The model may say "Oregon DEQ" on Monday and "Department of Environmental Quality" on Tuesday, and that is two groups. Every scan for a company is shown the labels already in use for that company and told to reuse them where they fit; new labels appear only when a document needs one. A person can merge two labels or rename one, and that is a fact for that customer. Later the requirement table's agency list becomes the canonical names and the merge happens by itself.

Unread files never vanish. Under any grouping, documents not yet read or not readable sit in their own group ("No agency yet"), never dropped because they have no value in the grouped column.

"Do they have everything" lives inside the by-agency view, labelled. Each agency group can end with one quiet line: "For an Oregon chemical distributor with employees we'd expect a Hazard Communication program here and don't see one. This is based on what similar companies hold, not on a checked requirement." Raw Claude writes it in rev 1. When the requirement table turns on, the same line becomes a finding with a citation and the label changes.

What a row shows. The scan's title in place of the filename ("Air Contaminant Discharge Permit 26-2841", not "Title_V_Air_Quality_Permit.pdf"), filename in the meta line with agency, kind, site and when we read it. On the right, the date that matters and the status word: amber for attention, grey for routine, never green for routine. Rows on the page background with hairlines, no boxes, 13px titles, 12px meta, per DESIGN.md.

Width. Reading surfaces stay at 775 (the workspace). Working surfaces, Documents, then Audits and Calendar, are 900: the row gets a real third column and Group by None becomes a proper table. The 224px sidebar leaves 78px of margin each side on a 1280 laptop, 121 at 1366, 158 at 1440. The drawer stays at 720. This goes into DESIGN.md as an addition; the layout chat owns that file.

Nested grouping (site then agency) is rev 2. Until then a two-site company picks the site in the filter.

Design reference. The canvas "CompliBoard Documents — folder layout options": artboards C and D are the page; E and F are the drawer. To be redrawn at 900 with the real sidebar.

The report drawer

Click a row and the report opens in a drawer from the right, 720 wide, over the dimmed page, exactly the workspace's drawer: 18px title, 13px sub line with kind, agency, site, filename and when it was read, hairlines instead of boxes, serif 17 for the summary, 16 for item names, 15 for descriptions, footer with one outlined button and text actions, Download at the far right, printable through the browser with a print header carrying company, title and date.

One drawer, and the sections appear by kind. A program shows gaps and drafts; a permit shows deadlines and conditions; a record shows what it records and how recent. These are on every report:

1. **Status and the date that matters.** "Needs work · 4 gaps · last revised February 2021", or "Expiring · 52 days left · expires 15 November 2026".
2. **Summary**, two or three sentences in the serif, saying what kind of thing this is and what that means. For a permit: "a permit is the agency's document, there is nothing in it to fix; what matters is renewing in time and meeting its conditions".
3. **The nudge**, in amber wash, when there is one: the freshness question, or an overdue renewal worked out from the permit's own condition, always with the way forward.
4. **What this document is.** Two columns of facts, each with where on the page it came from, and one link: "Not right? Change what this is".
5. **Gaps** (programs, policies). Numbered. Each has the plain description, a fix, the citation with a link, where in the document, and the actions: Draft this section where we can write it, Make a checklist, Research this, Not right. A checklist made earlier shows attached to its gap with progress: "Update the EAP contacts · checklist made 17 Sep · 2 of 5 done".
6. **Deadlines this document sets**, each with its source line and whether it is in the calendar; "Add to calendar" for the rest.
7. **Conditions to keep** (permits, certificates), with the condition number and the evidence you would hold, and "Add the log" where a record belongs beside it.
8. **Facts we found, please confirm.** The fact, the verbatim quote with its section, Confirm or Not right. The same rows appear in the To confirm section; confirming in either place confirms everywhere.
9. **Versions.** This version and the older ones, each with issue date and upload date, "Not the same permit?" on any proposed match.
10. **A quiet last line.** When it was read, that quotes are word for word, that inferences are marked as such, how many sources were looked up. Not legal advice.

Footer. Make a checklist for all N gaps (or Make a renewal checklist) outlined; Open the file; Read it again; Download.

Corrections. A person may change what the document is (kind, agency, site, date), confirm or reject a fact, say a version match is wrong, and say a gap is not right. Each correction outranks the model, is recorded as a fact for that customer, and is shown to the next scan of that document. The freedom is "you got that wrong, and here's why", not "doesn't apply".

Confirmations: one queue, one home

Facts found in documents are proposed, never written. This is the research section's rule (DECISIONS §108) applied to documents: nothing about a company is written silently. A scan that finds "42 employees at the Portland site" writes a proposal with the quote and the section; a person confirms it; only then is it a fact.

One queue for both sources. The research section already has `fact_proposals`: the nightly summariser reads quiet conversations and proposes facts with the sentence they came from; the workspace shows one at a time, "It sounded like X is Y", and Confirm writes it as a declared fact through the same route a person's own answer uses. Documents write to that same queue. A proposal from a document carries the document, the section and the quote; one from a conversation carries the turn and the sentence. No second queue.

The home is a new sidebar item: To confirm, with a count. It is the dashboard verification section from WORKSPACE.md §7.5 given its own door: three at a time, never more, ranked by what each answer unblocks, never by arrival order. The report drawer shows the confirmations that came from that document; To confirm shows all of them from every document and every conversation. A fact confirmed in either place is confirmed everywhere, because it is one row. Nobody confirms twice.

"Not right" is the one freedom, on facts and on gaps. It requires one line of reason. On a fact it rejects the proposal so it is not proposed again. On a gap it closes the gap as dismissed, never deleted, with the reason on the row; the next scan of that document reads the dismissed gaps and does not raise them again. When the reason is itself a fact about the company ("we have no critical operations to shut down"), it goes into the queue as a proposal from the person, the strongest kind of fact there is.

Two rules that keep the queue worth opening. The scan proposes only facts about the company, not everything true in the document; "the plan has seven sections" is not a fact worth a person's tap. And every proposal says in one line what it is for: "this affects which OSHA rules apply to you". In rev 1 both are the model's judgment, with a sentence in the prompt on what a company fact is; later the switch list decides.

Email falls out of the queue. A weekly note, "3 things to confirm, 2 permits expiring, 1 file we couldn't read", is a rendering of the queue plus the status groups, sent by the same cron and `job_runs` pattern the nightly jobs use, through Resend, which is already wired. The email has nothing of its own: it reads the same rows the page reads and every line links to its row. It can come after rev 1 with no rework.

Few versus many

A few files are read live, with progress that is real. The scan runs in the request, the person is waiting for the classification anyway, and the screen shows what is actually happening: "Reading the file", then "Searching: Oregon DEQ air permit renewal" because that is the search the model ran, then "Writing the report". The research section already streams the model's search events and counts them; a scan does the same. What the model didn't do is never shown. No scripted animation.

Many files go to the background. A folder upload of thirty or two hundred files cannot run in a request. Each document gets a status the screen shows truthfully while it waits (queued, reading, read, could not read); rows fill in as each one is read. When the run finishes the person gets an email and a banner in the app: "we've read 42 documents; 3 need work, 2 are expiring, 1 we couldn't read". Batch API at half price is the right path for overnight runs and is a measured switch after the baseline.

There is no worker yet, and rev 1 doesn't need one. CLAUDE.md §4 and BUILD-PLAN Phase 3 describe an always-on Node worker with `jobs` as its queue; the `jobs` table exists with zero rows and nothing reads or writes it. What exists is the research section's pattern: Vercel Cron hitting a protected route, one `job_runs` row per run with counts and per-item errors, one failure never stopping the sweep. A cron sweep over unread documents covers the many-files case. If a queue with progress a person can watch turns out to matter, that is the worker; if overnight is fine, it is cron. The brief decides; the schema is the same either way.

An unreadable file is a status, said out loud. "We could not read this clearly enough to rely on. A photo taken straight-on in good light, or the original PDF, would do it." On the row, on the report, and in the email. Never answered around, never dropped from a count. The audit engine today drops a document it cannot read with a console line and computes readiness from the rest; Documents ends that.

Cost is visible from the first scan. Every model call on the scan path writes an `ai_calls` row: model, effort, input and output tokens, searches, wall time, cost at the prices in `config/pricing.ts`. The first task of this section is exactly that, because a two-page PDF scan cost \$1.10 on production and nothing recorded why.

Deadlines, checklists and drafts

Deadlines go to the calendar on the person's say-so. The scan finds the dates a document sets, with the line they came from. On the report each one shows with "Add to calendar" or "In your calendar". For a single upload the existing popup, "dates found, send these to the calendar?", stays. For a folder upload there is no popup per file; the dates sit on each report and in the To confirm queue as a group, "14 dates found across 42 documents". Once a date is in the calendar, that section owns it. A calendar event carries the `document_id` it came from.

Every gap can become a checklist, and the checklist stays on the report. "Make a checklist" on a gap, or "Make a checklist for all N gaps" in the footer, uses the checklist engine the workspace already has. The checklist records which gap it came from, so a gap made today and acted on a week from now still shows "checklist made 17 Sep · 2 of 5 done" on its report, and the checklist drawer links back to the document. "Research this" opens the workspace with the gap as the question and the document attached.

Drafts where we can write them. The vision asks that we do everything to make the document compliant, including a draft of what should be there. Where a gap is a missing section whose content follows from the rest of the document, the report offers "Draft this section"; the draft

opens in the drawer for the person to copy. Where the fix needs something only they have, a floor plan, named roles, a number, the report says so plainly: "this needs your floor plan; we cannot draw it for you". A draft is never written into their document.

Rules that do not change

- **Best result first, with no conditions.** The baseline is the strongest model, one open call, search allowed, no shortcuts, however obvious. Every saving is added afterwards as a switch and measured on golden documents run three times.
- **Build on the cheapest model, judge on the real one.** `.env.local` runs Haiku. Output is judged on Opus 5 and Sonnet 5 only when the owner says.
- **Every model call writes a cost row.** No exceptions. Every Claude Code report ends with the session's cost from `npm run cost`.
- **Facts are proposed, never written silently.** Documents write to `fact_proposals`; a person confirms; only then is it a fact.
- **Unreadable is said out loud.** A file we cannot read is a status on the row, with the way forward, never a log line and never a count computed without it.
- **Close, never delete.** A dismissed gap, a rejected fact and an old version all stay, with the reason.
- **We scan first, then change the schema.** The contract follows the scan's output, not the other way round.
- **Honest words.** "Compliant" is never a status. "What we'd expect and don't see" is labelled as a suggestion until the requirement table makes it a finding. Progress shown on screen is what actually happened.
- **The word is "checklist".** Layout follows DESIGN.md.
- **Three roles stay separate.** Chat designs and writes one instruction at a time; Claude Code builds and tests on staging and never touches production; the owner decides product questions, runs production migrations and pushes. Every release follows RELEASE.md.

Later, not rev 1

- **Drive connection** (Google Drive, OneDrive): read in place, never download to keep, never rearrange, folders shown read-only. Its own run once the developer accounts exist.
- **The requirement table.** Once the baseline holds its own: `evidence_types` filled on the library so a document's kind joins to requirements; the "we'd expect and don't see" line becomes a finding; agency labels become canonical; `switch_determinations` and `obligation_evidence` start being written; the seven "never infer" rules from SWITCH-DETERMINATION.md come in as measured switches.
- **Chat over documents.** "Show me all training documents" answered from the reports; "am I compliant on wastewater" reads the reports first and opens the file only when needed. The research engine with a different context; the reports are the index.
- **Pinned views** beyond the default, and **nested grouping** (site then agency).

- **Measured savings:** cheap read and strong judge, no search during the read, search restricted to the agency's site, one pass, Batch API for overnight runs, prompt caching for documents re-sent per turn.
- **The weekly email** as a rendering of the queue and the status groups.
- **A model router** that picks the model by what the task needs, with a nearly free judge and its own golden checks.
- **Golden documents:** a few perfect and a few with gaps, across industries, built by us. Customer files do not enter the repo.

Where the code stands today, and the first task

The scan path, read from the code on 24 September. `lib/documentReview.ts` builds a PDF into a native document block, prepends file name, folder, division and industry, and calls `askAIJson` with `maxTokens: 6000` and web search on. No `task`, so it routes to `default`, which is `AI_MODEL || 'claude-sonnet-4-5'`; the bare `AI_MODEL` is not among the Vercel variables listed in `RELEASE.md`, so on production the scan most likely runs on Sonnet 4.5 at \$3/\$15. No `ledger`, no effort. A truncated answer is retried with a doubled budget, up to three attempts, each paid for. It has its own copy of file parsing, so Excel, CSV and text, which every picker accepts, fail on this path. The insert into `document_reviews` appends; one document can have many reviews and nothing marks which is current.

Three callers: the Documents page's Review button, the compliance page's attach flow, and the audit engine's auto-index loop, which on any failure logs and skips the document. The Documents page also calls `/api/extract-dates`, a second model call with no auth check (recorded in `TODO 0.8b`). Both calls are uncounted.

Tables that exist and matter: `documents`, `document_reviews`, `fact_proposals` (conversation-shaped: `topic_id` not null, no `document_id`), `job_runs`, `ai_calls` (task `document_review` already allowed), `turns` (with `attached_document_id` from migration 039), `switch_determinations` and `obligation_evidence` (empty, parked), `calendar_events` (no `document_id`), `company_folders` (zero rows on staging), `jobs` (zero rows, unread).

The first Claude Code task is measurement, not building. Part one: a cost row on every call on the scan path; no migration needed. Part two: scan the two-page fixture on staging on Haiku, read the rows back, price the same calls at the production model's prices, and say where the \$1.10 went. Hypotheses to check, not causes to assert: search iterations re-billing the whole context, reasoning-heavy output against the 6000 budget with a retry, and the event being two calls (the review plus the research turn that followed). The report ends with the call count per upload with the line that triggers each, one row per call, the priced reconstruction, the gap, and the session cost.

Then the Run 1 brief, written from this document: the open baseline scan with the contract, the schema changes, the page and the drawer from the design canvas, the To confirm item, cost rows,

unreadable as a status, two manual tests per feature in TESTING.md, decisions recorded in DECISIONS.md.

Open decisions and facts still needed

Decided on 24 September: upload only in this work, drive later; both versions kept; a document belongs to a site; email and banner when a background run finishes; synthetic golden documents across industries; folder as filter and group-by as arrangement, status first; 900 for working screens, 775 for reading, drawer at 720; "Not right" with a reason on gaps and facts, no "doesn't apply"; one confirmation queue with a To confirm sidebar item.

Facts the owner can supply and we cannot:

- Is the bare AI_MODEL set on Vercel Production? It decides which model the live scan ran on and what the reconstruction is priced at.
- In the Anthropic console, which model line did the \$1.10 sit under, and was it one call or two (the review plus the research turn that followed)?
- Are the six 3,852-byte files on production one demo FMLA handbook uploaded under six names? If yes, SWITCH-DETERMINATION.md §8.2 and TESTING.md are corrected: the scan was right and the "strongest negative control" does not exist.

Still to decide, in the brief:

- Cron sweep or the real worker for folder uploads. Recommendation: cron, adopt into the worker later if it arrives.
- Whether subject is a rev 1 grouping or waits until we have seen what labels the scans produce.
- What counts as "many": the file count above which an upload goes to the background rather than the request.
- The size limit, if any, for a document carried on every turn of a conversation. Prompt caching moves up the list because of this.
- The prototype convention: the design canvas is the reference; whether its artboards are exported into `prototypes/` for Claude Code to read, or a screenshot set is committed instead.

Known debts touching this section, not fixed here: `/api/extract-dates` has no auth check (TODO 0.8b, waits on M7's pre-login auth story); the audit engine's silent drop (M2) ends when it reads the document index instead of re-reviewing files; `document_reviews` rows on production (33) are re-extracted through the new scan, never mined.